Student Attendance Management

using a Dynamically Resizable Matrix in C

Name : Rudra Pratap Singh

SAP ID : 590032811

Batch : 16

1. Problem Statement

Develop a C program to manage student attendance using a dynamically resizable matrix. Each row of the matrix represents one student and each column represents one working day. The value 1 indicates Present and 0 indicates Absent. The program must use pointers and dynamic memory allocation, and must not use a fixed-size 2D array.

2. Requirements Covered

- Accepts the initial number of students and working days.
- Uses malloc() to allocate the initial attendance matrix.
- Accepts attendance of the initial students (only 0 or 1 is allowed).
- Allows more students to be added at run time; realloc() is used to enlarge the matrix.
- Calculates and displays the attendance percentage of every student.
- Uses separate functions for each task.
- Handles invalid input and memory-allocation failure.
- Releases all allocated memory with free() before the program ends.

3. Approach

Instead of a fixed 2D array, the matrix is stored as a single block of memory on the heap and accessed through an int pointer. Because the block is contiguous, the element for student i on day j is found using pointer arithmetic:

```
*(matrix + i * days + j)
```

Here i is the row (student) index starting from 0, j is the column (day) index starting from 0, and days is the number of columns. When new students are added, only the number of rows changes, so a single realloc() call can extend the block to (students + extra) * days integers. Existing data is preserved by realloc(), and only the new rows are initialised and filled.

4. Algorithm

1. Start. Read the number of students and the number of working days (with validation).
2. Allocate memory for students × days integers using malloc(). If it returns NULL, display an error and stop.
3. For every student and every day, read the attendance value and accept only 0 or 1.
4. Display the attendance matrix.
5. Ask how many students are to be added. If the answer is 0, go to step 8.
6. Use realloc() to enlarge the block. Use a temporary pointer so the old data is not lost if realloc() fails.
7. Read attendance for the new students only, display the updated matrix, and go back to step 5.

8. For each student, count the days present and compute (present ÷ days) × 100; display the result.
9. Release the memory using free() and set the pointer to NULL. Stop.

5. Functions Used

Function	Purpose
readInt()	Reads an integer within a given range and keeps asking until the input is valid.
createMatrix()	Allocates the initial matrix with malloc() and checks for failure.
recordAttendance()	Reads attendance (0 or 1) for a range of students.
addStudents()	Enlarges the matrix with realloc() and records attendance of the new students.
displayAttendance()	Prints the attendance matrix in table form.
calculatePercentages()	Calculates and prints the attendance percentage of each student.
releaseMemory()	Frees the allocated memory and resets the pointer to NULL.

6. Source Code

File name: attendance.c

```
/*
 * HDOC Day-72 Activity : Dynamic Student Attendance Matrix
 * Name   : Rudra Pratap Singh
 * SAP ID : 590032811
 * Batch  : 16
 *
 * Rows    -> students
 * Columns -> working days
 * 1 = Present, 0 = Absent
 *
 * The matrix is stored as one block of heap memory and accessed
 * only through pointers (no fixed-size 2D array is used).
 * Element (i, j) is  *(matrix + i * days + j).
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BUFFER_SIZE 64

/* ---------- Function prototypes ---------- */
int   readInt(const char *prompt, int min, int max);
int  *createMatrix(int students, int days);
void  recordAttendance(int *matrix, int fromStudent, int toStudent, int days);
int   addStudents(int **matrix, int *students, int days, int extra);
void  displayAttendance(const int *matrix, int students, int days);
void  calculatePercentages(const int *matrix, int students, int days);
void  releaseMemory(int **matrix);

/* Reads one integer in [min, max]; keeps asking until input is valid. */
int readInt(const char *prompt, int min, int max)
{
    char buffer[BUFFER_SIZE];
    char *end;
    long value;
```

```
    while (1)
    {
        printf("%s", prompt);

        if (fgets(buffer, sizeof(buffer), stdin) == NULL)
        {
            printf("\nInput ended unexpectedly. Exiting.\n");
            exit(EXIT_FAILURE);
        }

        value = strtol(buffer, &end, 10);

        /* reject empty input, extra characters, and out-of-range values */
        if (end == buffer || (*end != '\n' && *end != '\0'))
            printf("  Invalid input. Please enter a whole number.\n");
        else if (value < min || value > max)
        {
            if (min == 0 && max == 1)
                printf("  Invalid input. Enter only 0 or 1.\n");
            else
                printf("  Invalid input. Enter a value between %d and %d.\n", min, max);
        }
        else
            return (int)value;
    }
}

/* Allocates the initial students x days matrix with malloc(). */
int *createMatrix(int students, int days)
{
    int *matrix = (int *)malloc((size_t)students * days * sizeof(int));

    if (matrix == NULL)
    {
        printf("Error: memory allocation failed.\n");
        return NULL;
    }

    memset(matrix, 0, (size_t)students * days * sizeof(int));
    return matrix;
}

/* Records attendance (0 or 1 only) for students fromStudent .. toStudent-1. */
void recordAttendance(int *matrix, int fromStudent, int toStudent, int days)
{
    int i, j;
    char prompt[BUFFER_SIZE];

    for (i = fromStudent; i < toStudent; i++)
    {
        printf("\nStudent %d:\n", i + 1);
        for (j = 0; j < days; j++)
        {
            sprintf(prompt, "  Day %d (1 = Present, 0 = Absent): ", j + 1);
            *(matrix + i * days + j) = readInt(prompt, 0, 1);
        }
    }
}

/* Grows the matrix with realloc() to hold 'extra' more students. */
int addStudents(int **matrix, int *students, int days, int extra)
{
    int newTotal = *students + extra;

    /* use a temporary pointer so the old block is not lost if realloc fails */
    int *temp = (int *)realloc(*matrix, (size_t)newTotal * days * sizeof(int));
```

```
    if (temp == NULL)
    {
        printf("Error: memory reallocation failed. No students were added.\n");
        return 0;
    }

    *matrix = temp;

    /* initialise the newly added rows */
    memset(*matrix + (*students) * days, 0, (size_t)extra * days * sizeof(int));

    recordAttendance(*matrix, *students, newTotal, days);
    *students = newTotal;
    return 1;
}

/* Prints the attendance matrix as a table. */
void displayAttendance(const int *matrix, int students, int days)
{
    int i, j;

    printf("\n%-12s", "Student");
    for (j = 0; j < days; j++)
        printf("Day %-4d", j + 1);
    printf("\n");

    for (i = 0; i < students; i++)
    {
        printf("Student %-4d", i + 1);
        for (j = 0; j < days; j++)
            printf("%-8d", *(matrix + i * days + j));
        printf("\n");
    }
}

/* Calculates and prints each student's attendance percentage. */
void calculatePercentages(const int *matrix, int students, int days)
{
    int i, j, present;
    double percentage;

    printf("\nAttendance Percentage:\n");
    for (i = 0; i < students; i++)
    {
        present = 0;
        for (j = 0; j < days; j++)
            present += *(matrix + i * days + j);

        percentage = ((double)present / days) * 100.0;
        printf("Student-%d: %.1f%%\n", i + 1, percentage);
    }
}

/* Frees the matrix and resets the pointer to NULL. */
void releaseMemory(int **matrix)
{
    if (*matrix != NULL)
    {
        free(*matrix);
        *matrix = NULL;
    }
}

int main(void)
{
    int *attendance = NULL;
    int students, days, extra;
```

```
    printf("===== Student Attendance Management =====\n\n");

    students = readInt("Enter initial number of students : ", 1, 10000);
    days     = readInt("Enter number of working days     : ", 1, 366);

    attendance = createMatrix(students, days);
    if (attendance == NULL)
        return EXIT_FAILURE;

    recordAttendance(attendance, 0, students, days);

    printf("\n--- Initial Attendance Matrix ---");
    displayAttendance(attendance, students, days);

    /* keep adding students until the user enters 0 */
    while (1)
    {
        extra = readInt("\nEnter number of students to add (0 to stop): ", 0, 10000);
        if (extra == 0)
            break;

        if (addStudents(&attendance, &students, days, extra))
        {
            printf("\n--- Updated Attendance Matrix ---");
            displayAttendance(attendance, students, days);
        }
    }

    calculatePercentages(attendance, students, days);

    releaseMemory(&attendance);
    printf("\nMemory released. Program ended.\n");
    return EXIT_SUCCESS;
}
```

7. Sample Output

7.1 Example from the problem (3 students, 3 days, then 3 more students)

Values typed by the user appear after each prompt.

```
===== Student Attendance Management =====

Enter initial number of students : 3
Enter number of working days     : 3

Student 1:
  Day 1 (1 = Present, 0 = Absent): 1
  Day 2 (1 = Present, 0 = Absent): 0
  Day 3 (1 = Present, 0 = Absent): 0

Student 2:
  Day 1 (1 = Present, 0 = Absent): 1
  Day 2 (1 = Present, 0 = Absent): 0
  Day 3 (1 = Present, 0 = Absent): 1

Student 3:
  Day 1 (1 = Present, 0 = Absent): 1
  Day 2 (1 = Present, 0 = Absent): 1
  Day 3 (1 = Present, 0 = Absent): 1

--- Initial Attendance Matrix ---
Student     Day 1   Day 2   Day 3
Student 1   1       0       0
Student 2   1       0       1
Student 3   1       1       1

Enter number of students to add (0 to stop): 3

Student 4:
  Day 1 (1 = Present, 0 = Absent): 1
  Day 2 (1 = Present, 0 = Absent): 0
  Day 3 (1 = Present, 0 = Absent): 0

Student 5:
  Day 1 (1 = Present, 0 = Absent): 1
  Day 2 (1 = Present, 0 = Absent): 1
  Day 3 (1 = Present, 0 = Absent): 1

Student 6:
  Day 1 (1 = Present, 0 = Absent): 1
  Day 2 (1 = Present, 0 = Absent): 0
  Day 3 (1 = Present, 0 = Absent): 1

--- Updated Attendance Matrix ---
Student     Day 1   Day 2   Day 3
Student 1   1       0       0
Student 2   1       0       1
Student 3   1       1       1
Student 4   1       0       0
Student 5   1       1       1
Student 6   1       0       1

Enter number of students to add (0 to stop): 0

Attendance Percentage:
Student-1: 33.3%
Student-2: 66.7%
Student-3: 100.0%
Student-4: 33.3%
```

```
Student-5: 100.0%
Student-6: 66.7%

Memory released. Program ended.
```

The matrix and percentages match the expected result given in the problem statement: 33.3%, 66.7%, 100%, 33.3%, 100% and 66.7%.

7.2 Invalid input handling

Text, zero, negative numbers and values other than 0 or 1 are rejected and the user is asked again.

```
===== Student Attendance Management =====

Enter initial number of students : abc
  Invalid input. Please enter a whole number.
Enter initial number of students : 0
  Invalid input. Enter a value between 1 and 10000.
Enter initial number of students : 2
Enter number of working days     : -1
  Invalid input. Enter a value between 1 and 366.
Enter number of working days     : 3

Student 1:
  Day 1 (1 = Present, 0 = Absent): 1
  Day 2 (1 = Present, 0 = Absent): 5
  Invalid input. Enter only 0 or 1.
  Day 2 (1 = Present, 0 = Absent): x
  Invalid input. Please enter a whole number.
  Day 2 (1 = Present, 0 = Absent): 0
  Day 3 (1 = Present, 0 = Absent): 1

Student 2:
  Day 1 (1 = Present, 0 = Absent): 1
  Day 2 (1 = Present, 0 = Absent): 1
  Day 3 (1 = Present, 0 = Absent): 0

--- Initial Attendance Matrix ---
Student     Day 1   Day 2   Day 3
Student 1   1       0       1
Student 2   1       1       0

Enter number of students to add (0 to stop): 0

Attendance Percentage:
Student-1: 66.7%
Student-2: 66.7%

Memory released. Program ended.
```

8. Memory Management

- malloc() is used once to create the initial matrix; its return value is checked for NULL.
- realloc() is assigned to a temporary pointer first. If it fails, the original matrix remains valid and no data is lost.
- free() is called through releaseMemory() before the program terminates, and the pointer is set to NULL.

- The program was compiled with gcc -Wall -Wextra without warnings and tested with AddressSanitizer/LeakSanitizer, which reported no memory leaks or errors.

9. Conclusion

The program manages attendance with a matrix that grows at run time. It uses malloc() for the initial allocation, realloc() for resizing, pointer arithmetic for access, and free() for cleanup. Every input is validated, allocation failures are handled safely, and the attendance percentages are calculated correctly. This activity gave practice in dynamic memory allocation, pointer arithmetic, modular programming with functions, and input validation in C.